

Data Preparation

```
In [2]: # importing Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [203... # Loading datasets
ratings_df = pd.read_csv(r"C:\Users\hp\Desktop\Muha 2025\Dataset1\ml-latest-small\m
movies_df = pd.read_csv(r"C:\Users\hp\Desktop\Muha 2025\Dataset1\ml-latest-small\ml
tags_df = pd.read_csv(r"C:\Users\hp\Desktop\Muha 2025\Dataset1\ml-latest-small\ml-l
links_df = pd.read_csv(r"C:\Users\hp\Desktop\Muha 2025\Dataset1\ml-latest-small\ml-l
print(ratings_df.shape, movies_df.shape, tags_df.shape, links_df.shape)
```

```
(100836, 4) (9742, 3) (3683, 4) (9742, 3)
```

```
In [205... # Merging datasets
df = ratings_df.merge(movies_df, on='movieId', how='left')
df.head()
```

```
Out[205...   userId  movieId  rating  timestamp      title  genre
0         1         1     4.0   964982703  Toy Story (1995)  Adventure|Animation|Children|Comedy|Fantas
1         1         3     4.0   964981247  Grumpier Old Men (1995)  Comedy|Romanc
2         1         6     4.0   964982224    Heat (1995)  Action|Crime|Thrille
3         1        47     5.0   964983815  Seven (a.k.a. Se7en) (1995)  Mystery|Thrille
4         1        50     5.0   964982931  Usual Suspects, The (1995)  Crime|Mystery|Thrille
```



Data Cleaning

```
In [6]: # Duplicates
duplicates = df.duplicated().sum()
duplicates
```

Out[6]: 0

```
In [7]: # Missing values
missing_values = df.isnull().sum()
missing_values
```

```
Out[7]: userId      0
movieId      0
rating       0
timestamp    0
title        0
genres       0
dtype: int64
```

```
In [207... # Handling Missing Values
df['genres'] = df['genres'].replace('(no genres listed)', 'Unknown')
df.head()
```

```
Out[207... 
```

	userId	movieId	rating	timestamp	title	genre
0	1	1	4.0	964982703	Toy Story (1995)	Adventure Animation Children Comedy Fantas
1	1	3	4.0	964981247	Grumpier Old Men (1995)	Comedy Romanc
2	1	6	4.0	964982224	Heat (1995)	Action Crime Thrille
3	1	47	5.0	964983815	Seven (a.k.a. Se7en) (1995)	Mystery Thrille
4	1	50	5.0	964982931	Usual Suspects, The (1995)	Crime Mystery Thrille

```
In [9]: # Convert Timestamp to datetime

df['timestamp'] = pd.to_datetime(df['timestamp'], unit = 's')
df['timestamp']
```

```
Out[9]: 0      2000-07-30 18:45:03
        1      2000-07-30 18:20:47
        2      2000-07-30 18:37:04
        3      2000-07-30 19:03:35
        4      2000-07-30 18:48:51
        ...
        100831 2017-05-03 21:53:22
        100832 2017-05-03 22:21:31
        100833 2017-05-08 19:50:47
        100834 2017-05-03 21:19:12
        100835 2017-05-03 21:20:15
Name: timestamp, Length: 100836, dtype: datetime64[ns]
```

Feature Engineering

```
In [33]: # Release year
df['release_year'] = df['title'].str.extract(r'\((\d{4})\)').fillna(0).astype(int)

df
```

Out[33]:

	userId	movieId	rating	timestamp	title	
0	1	1	4.0	2000-07-30 18:45:03	Toy Story (1995)	Adventure Animation Children Comedy
1	1	3	4.0	2000-07-30 18:20:47	Grumpier Old Men (1995)	Comedy Romance
2	1	6	4.0	2000-07-30 18:37:04	Heat (1995)	Action Crime
3	1	47	5.0	2000-07-30 19:03:35	Seven (a.k.a. Se7en) (1995)	Mystery Thriller
4	1	50	5.0	2000-07-30 18:48:51	Usual Suspects, The (1995)	Crime Mystery Thriller
...	...	...	...	...	...	...
100831	610	166534	4.0	2017-05-03 21:53:22	Split (2017)	Drama Horror Thriller
100832	610	168248	5.0	2017-05-03 22:21:31	John Wick: Chapter Two (2017)	Action Crime Thriller
100833	610	168250	5.0	2017-05-08 19:50:47	Get Out (2017)	Horror Thriller
100834	610	168252	5.0	2017-05-03 21:19:12	Logan (2017)	Action Drama
100835	610	170875	3.0	2017-05-03 21:20:15	The Fate of the Furious (2017)	Action Crime Drama

100836 rows × 11 columns



In [158...]

```
# num_genres
df['num_genres'] = df['genres'].apply(lambda x: len(x.split('|'))) if pd.notnull(x)
df
```

Out[158...

	userId	movieId	rating	timestamp	title	
0	1	1	4.0	2000-07-30 18:45:03	Toy Story (1995)	Adventure Animation Children Comedy
1	1	3	4.0	2000-07-30 18:20:47	Grumpier Old Men (1995)	Comedy Romance
2	1	6	4.0	2000-07-30 18:37:04	Heat (1995)	Action Crime
3	1	47	5.0	2000-07-30 19:03:35	Seven (a.k.a. Se7en) (1995)	Mystery Thriller
4	1	50	5.0	2000-07-30 18:48:51	Usual Suspects, The (1995)	Crime Mystery Thriller
...	...	...	...	...	...	...
100831	610	166534	4.0	2017-05-03 21:53:22	Split (2017)	Drama Horror Thriller
100832	610	168248	5.0	2017-05-03 22:21:31	John Wick: Chapter Two (2017)	Action Crime Thriller
100833	610	168250	5.0	2017-05-08 19:50:47	Get Out (2017)	Horror Thriller
100834	610	168252	5.0	2017-05-03 21:19:12	Logan (2017)	Action Drama
100835	610	170875	3.0	2017-05-03 21:20:15	The Fate of the Furious (2017)	Action Crime Drama

100836 rows × 12 columns



In [160...

```
# top_genre
df['top_genre'] = df['genres'].apply(lambda x: x.split('|')[0] if isinstance(x, str) else x)
df[['genres', 'top_genre']]
```

Out[160...

	genres	top_genre
0	Adventure Animation Children Comedy Fantasy	Adventure
1	Comedy Romance	Comedy
2	Action Crime Thriller	Action
3	Mystery Thriller	Mystery
4	Crime Mystery Thriller	Crime
...	...	...
100831	Drama Horror Thriller	Drama
100832	Action Crime Thriller	Action
100833	Horror	Horror
100834	Action Sci-Fi	Action
100835	Action Crime Drama Thriller	Action

100836 rows × 2 columns

```
In [67]: # avg_movie_rating
avg_movie_rating = df.groupby('movieId')['rating'].mean().reset_index().rename(column

df = df.merge(avg_movie_rating, on='movieId', how='left')

df[['movieId', 'avg_movie_rating']].head()

print(df[['movieId', 'rating', 'avg_movie_rating']].head())
```

	movieId	rating	avg_movie_rating
0	1	4.0	3.920930
1	3	4.0	3.259615
2	6	4.0	3.946078
3	47	5.0	3.975369
4	50	5.0	4.237745

```
In [63]: # avg_user_rating
avg_user_rating = df.groupby('userId')['rating'].mean().reset_index().rename(column

df = df.merge(avg_user_rating, on='userId', how='left')

df[['userId', 'avg_user_rating']].head()
```

Out[63]:

	userId	avg_user_rating
0	1	4.366379
1	1	4.366379
2	1	4.366379
3	1	4.366379
4	1	4.366379

In [79]:

```
# age_of_movie

from datetime import datetime

current_year = datetime.now().year

df['age_of_movie'] = current_year - df['release_year']

df[['title', 'age_of_movie']]
```

Out[79]:

	title	age_of_movie
0	Toy Story (1995)	30
1	Grumpier Old Men (1995)	30
2	Heat (1995)	30
3	Seven (a.k.a. Se7en) (1995)	30
4	Usual Suspects, The (1995)	30
...	...	...
100831	Split (2017)	8
100832	John Wick: Chapter Two (2017)	8
100833	Get Out (2017)	8
100834	Logan (2017)	8
100835	The Fate of the Furious (2017)	8

100836 rows × 2 columns

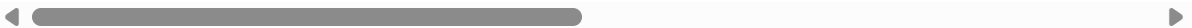
In [87]:

```
df = df.drop(['avg_movie_rating_x', 'avg_user_rating_x', 'avg_user_rating_y', 'av
df
```

Out[87]:

	userId	movieId	rating	timestamp	title	
0	1	1	4.0	2000-07-30 18:45:03	Toy Story (1995)	Adventure Animation Children Comedy
1	1	3	4.0	2000-07-30 18:20:47	Grumpier Old Men (1995)	Comedy R
2	1	6	4.0	2000-07-30 18:37:04	Heat (1995)	Action Crime
3	1	47	5.0	2000-07-30 19:03:35	Seven (a.k.a. Se7en) (1995)	Mystery
4	1	50	5.0	2000-07-30 18:48:51	Usual Suspects, The (1995)	Crime Mystery
...	...	...	...	...	...	
100831	610	166534	4.0	2017-05-03 21:53:22	Split (2017)	Drama Horror
100832	610	168248	5.0	2017-05-03 22:21:31	John Wick: Chapter Two (2017)	Action Crime
100833	610	168250	5.0	2017-05-08 19:50:47	Get Out (2017)	
100834	610	168252	5.0	2017-05-03 21:19:12	Logan (2017)	Action
100835	610	170875	3.0	2017-05-03 21:20:15	The Fate of the Furious (2017)	Action Crime Drama

100836 rows × 12 columns



Exploratory data Analysis

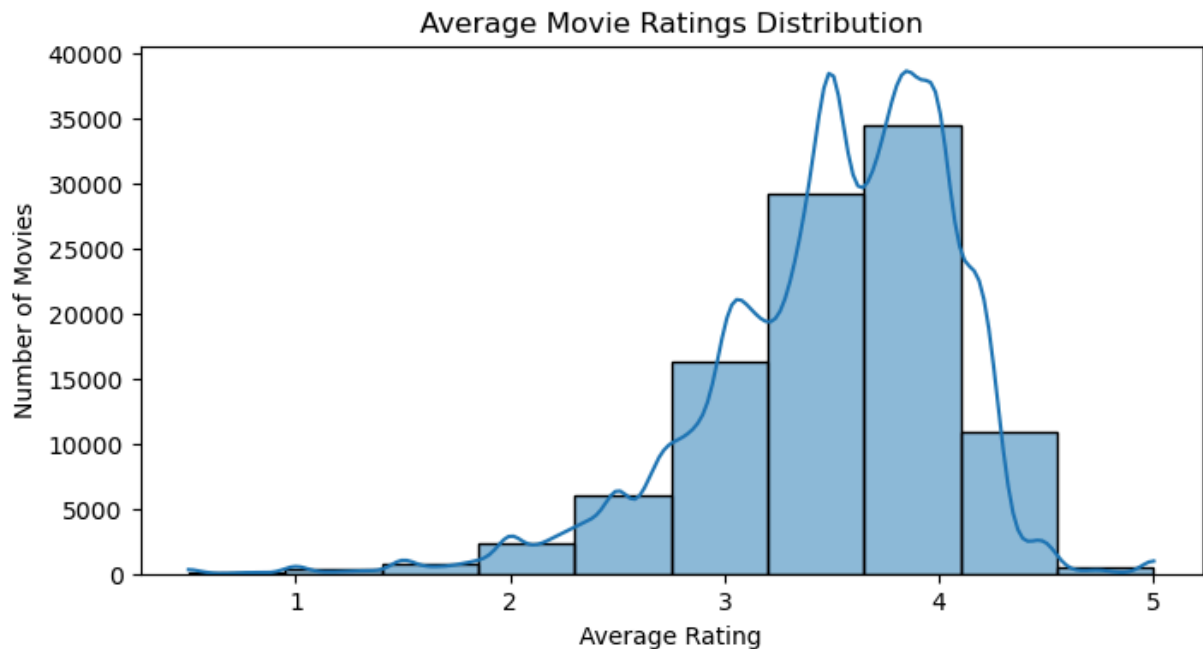
In [183...

```
# Average Movie Ratings Distribution

plt.figure(figsize=(8,4))
sns.histplot(df['avg_movie_rating'], bins=10, kde=True)
plt.title("Average Movie Ratings Distribution")
plt.xlabel("Average Rating")
```

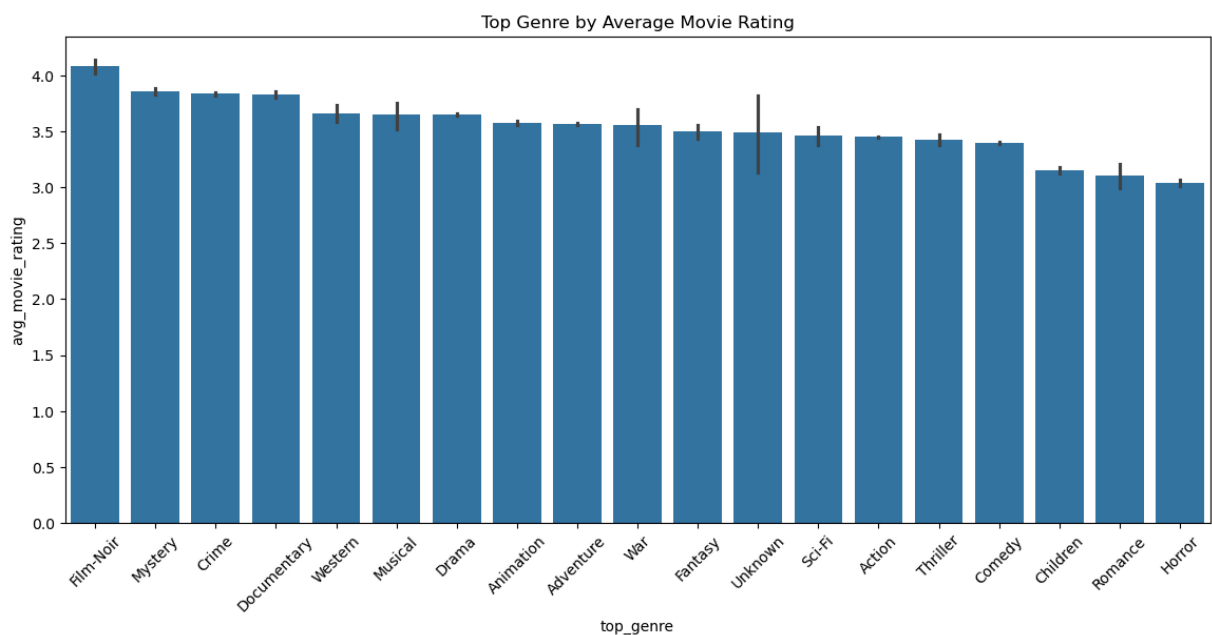


```
plt.ylabel("Number of Movies")
plt.savefig("Average_rovie_ratings_distribution.png", dpi=300, bbox_inches="tight")
plt.show()
```



```
In [185... # Top Genre Analysis

plt.figure(figsize=(14,6))
sns.barplot(data=df, x='top_genre', y='avg_movie_rating', order=df.groupby('top_gen
plt.xticks(rotation=45)
plt.title("Top Genre by Average Movie Rating")
plt.savefig("top_genre.png", dpi=300, bbox_inches="tight")
plt.show()
```



```
In [193... # Most Rated movies
```

```
topRated_movies = df.sort_values('avg_movie_rating', ascending=False).head(10)[['t
topRated_movies
```

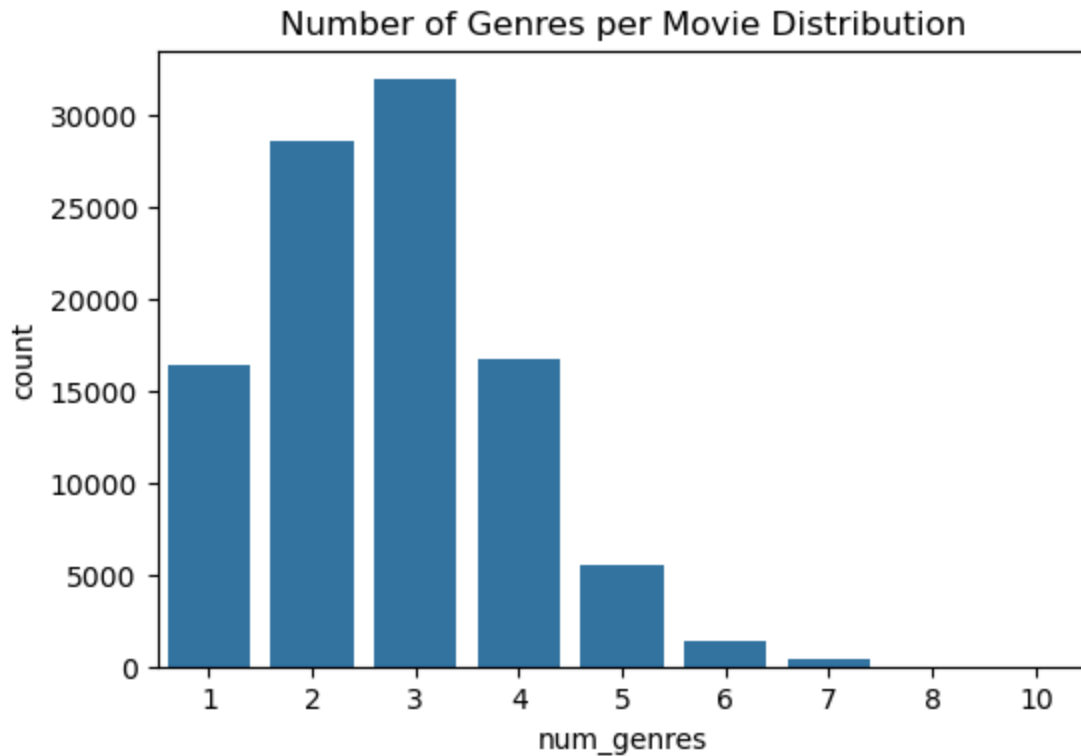
Out[193...

	title	avg_movie_rating	num_genres	age_of_movie
66992	Peaceful Warrior (2006)	5.0	1	19
16936	Loving Vincent (2017)	5.0	3	8
16885	The Girls (1961)	5.0	2	64
64142	Belle époque (1992)	5.0	2	33
17249	Rivers and Tides (2001)	5.0	1	24
28004	What Happened Was... (1994)	5.0	4	31
68366	Odd Life of Timothy Green, The (2012)	5.0	3	13
78207	Tyler Perry's I Can Do Bad All by Myself (2009)	5.0	2	16
16886	The Adventures of Sherlock Holmes and Doctor W...	5.0	1	2025
16917	Vacations in Prostokvashino (1980)	5.0	1	45

In [195...

```
# Number of Genre Distribution
```

```
plt.figure(figsize=(6,4))
sns.countplot(data=df, x='num_genres')
plt.title("Number of Genres per Movie Distribution")
plt.savefig("num_genre.png", dpi=300, bbox_inches="tight")
plt.show()
```



In [197...

```

# Average Rating vs Age of movie

# creating copy
filtered_df = filtered_df.copy()

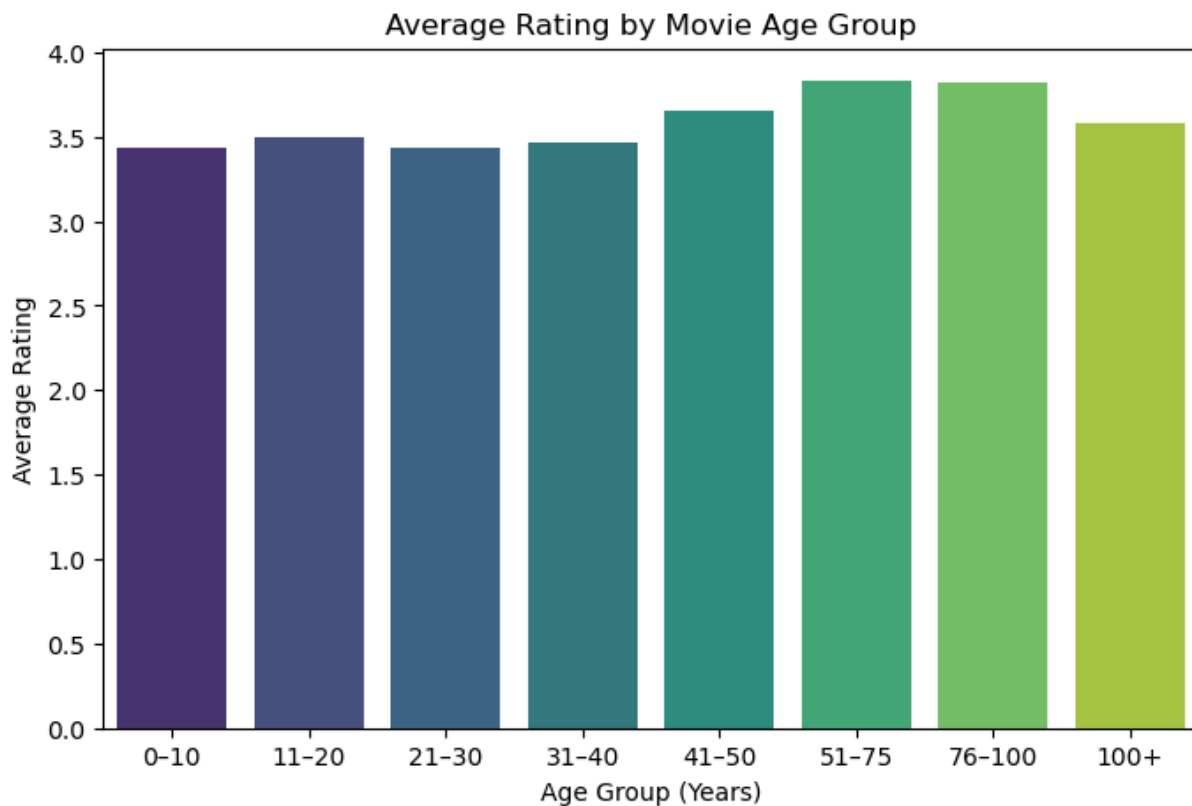
# Create age groups
bins = [0, 10, 20, 30, 40, 50, 75, 100, 150]
labels = ['0-10', '11-20', '21-30', '31-40', '41-50', '51-75', '76-100', '100+']

filtered_df['age_group'] = pd.cut(filtered_df['age_of_movie'], bins=bins, labels=labels)

age_grouped = filtered_df.groupby('age_group', observed=True)['avg_movie_rating'].mean()

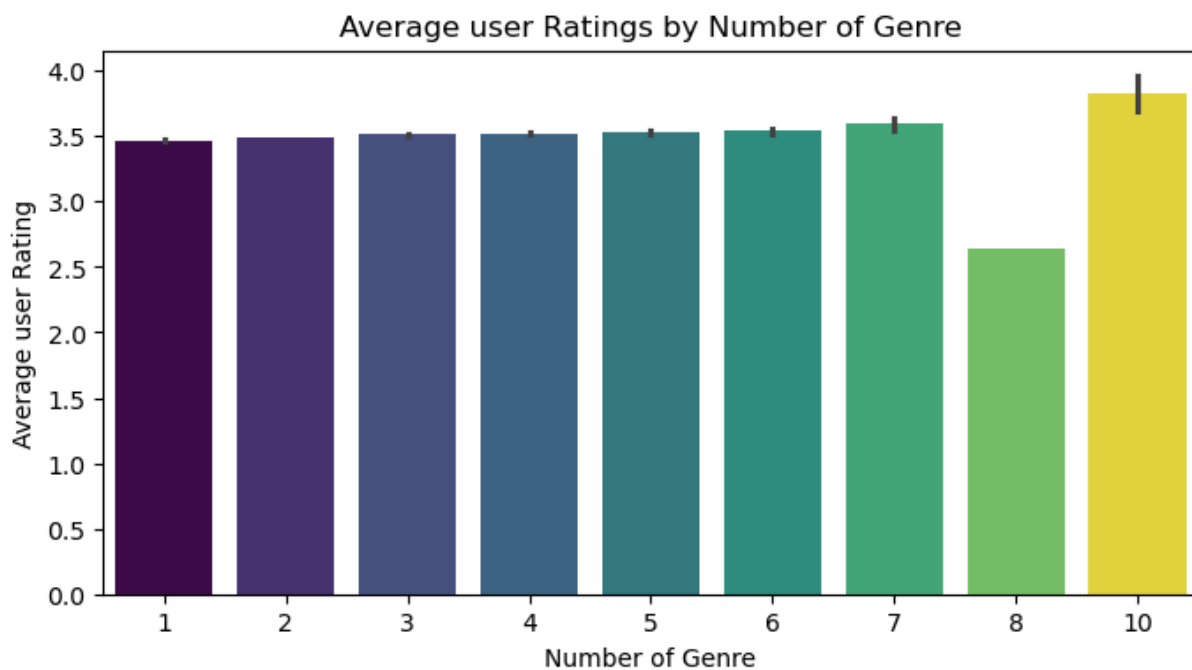
plt.figure(figsize=(8,5))
sns.barplot(data=age_grouped, x='age_group', y='avg_movie_rating', hue='age_group',
plt.title("Average Rating by Movie Age Group")
plt.xlabel("Age Group (Years)")
plt.ylabel("Average Rating")
plt.savefig("avg_rating.png", dpi=300, bbox_inches="tight")
plt.show()

```



In [201...

```
# number of genre vs average rating
plt.figure(figsize=(8,4))
sns.barplot(x='num_genres', y='avg_user_rating', data=df, hue='num_genres', palette=
plt.title("Average user Ratings by Number of Genre")
plt.xlabel("Number of Genre")
plt.ylabel("Average user Rating")
plt.savefig("avg_user_rating.png", dpi=300, bbox_inches="tight")
plt.show()
```



```
In [156... df.to_csv('df.csv', index=False)
```

```
In [ ]:
```